ITERATION LAB

DESIGN · SIMULATE · MACHINE

RESEARCH PAPER · 2026

THE RESEARCH QUESTION

Two Perfect Parts That Refuse to Fit Together

How individually conforming components fail to assemble when dimensional tolerances accumulate



MD Ikramul Hoque (Arik)

Researcher · Maker · CNC Programmer

BKTTC SICIP CNC Trainee · Chattogram, Bangladesh

The Iteration Lab · Design · Simulate · Machine

Contents

Abstract . . . 2
1. Introduction . . . 3
2. Background: Two Ways to Add Tolerances . . . 3
3. Method . . . 4
4. Results . . . 5
5. Discussion: Why “Perfect” Parts Fail . . . 6
6. Worked Example, Step by Step . . . 7
7. How to Prevent It . . . 8
8. The Software: Tolerance Stack-Up Simulator . . . 9
9. Limitations, Future Work and Conclusion . . . 12
References . . . 12
About the Author . . . 13

Key findings at a glance

0.30 mm	0.01 %	65 %	0
nominal clearance in the stack	assemblies failing, centered process	failing when the process drifts +0.08 mm	parts ever out of tolerance

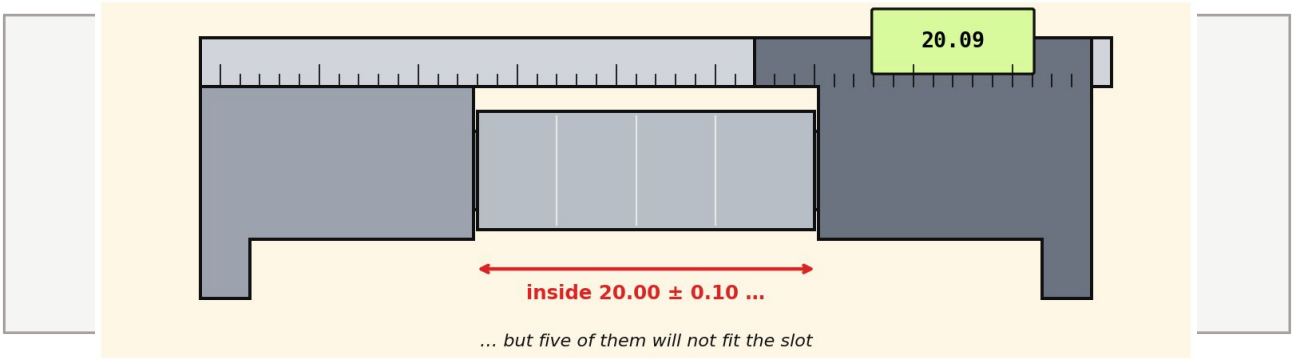
Abstract

Every part on the table passes inspection. Every dimension sits inside its tolerance band. Yet when the parts are stacked into their housing, the assembly jams. This paper explains why.

Using a simple, realistic example (five 20 mm parts in a 100.30 mm slot), I compare the two classical stack-up methods, **worst-case** and **root-sum-square (RSS)**, and then test them against a **one-million-assembly Monte Carlo simulation**.

The result: a drawing can be perfectly respected and still produce an assembly that fails, up to **65% of the time** when the manufacturing process quietly drifts toward one side of the tolerance band. Tolerances are not individual promises. They are a shared budget, and the budget is spent in the *sum*.

Keywords: tolerance stack-up, worst-case analysis, RSS, Monte Carlo, process drift, fits and clearances, CNC machining



1. Introduction

A tolerance is a permission: *“this dimension may be anywhere between these two limits.”* One part with a ±0.10 mm tolerance is easy to judge, because it is either inside the band or outside it.

The trouble begins at **assembly**, where dimensions are added, subtracted and stacked in a chain. Each link in the chain is allowed to wander a little, and the wandering **accumulates**. The final gap that decides whether the product works is called the **closing dimension**, and its variation is the combined variation of everything in the loop.

> ***Core idea:*** *Parts are inspected one at a time, but they fail together.*

This is a daily reality in machining. A turned shaft, a milled slot and a bored housing can each be “in tolerance” on the inspection sheet and still refuse to go together on the bench.

In this paper the problem is made as small and clear as possible: one straight chain of identical parts in one slot. Small as it is, it contains everything that goes wrong in larger assemblies.

2. Background: Two Ways to Add Tolerances

2.1 Worst-case (WC) analysis

Assume **every dimension lands at its limit in the worst possible direction at the same time**. The tolerances simply add:

$$T_{WC} = \Sigma\, T_i$$

This **guarantees** that every assembly fits, but it forces very tight (expensive) tolerances, and it describes an event that almost never happens in practice.

2.2 Root-sum-square (RSS) analysis

Assume each dimension varies **randomly and independently** around its center. Small errors partly cancel, so the total grows more slowly:

$$T_{RSS} = \sqrt{\left(\Sigma\, T_i^{\,2}\right)}$$

RSS is far more optimistic, but it depends on two assumptions: the process is **centered** and the errors are **independent**. Both are easy to break on a real shop floor (see Section 5).

2.3 The stack studied here

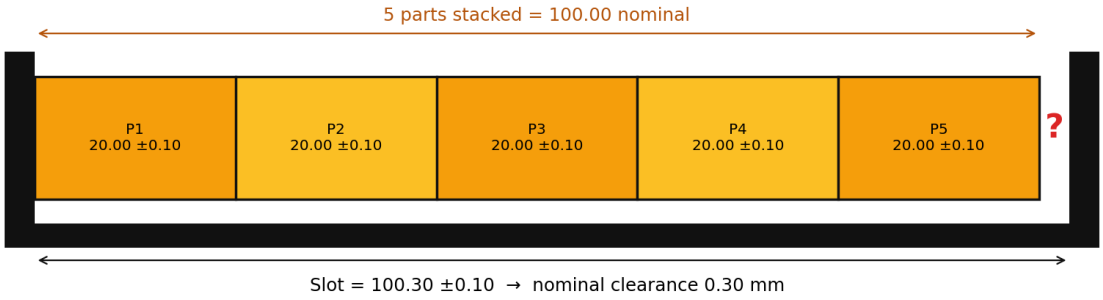


Figure 1. Five 20.00 ± 0.10 mm parts in a 100.30 ± 0.10 mm slot. Nominal clearance is only 0.30 mm.

3. Method

The study has three layers, so each result can be cross-checked against the others.

Layer 1: Analytical. Worst-case and RSS clearance limits for the stack in Figure 1.

Layer 2: Monte Carlo simulation. One million virtual assemblies per scenario. Each part is drawn from a normal distribution with σ = T / 3 (process capability Cp = 1). Only assemblies whose five parts **all pass inspection** are kept, so the simulation never “cheats” with out-of-tolerance parts.

Layer 3: Scenarios. Six different ways the factory might behave:

Scenario	What it represents
Centered process	Ideal shop: every part averages exactly 20.00 mm
Uniform across band	Parts spread evenly anywhere inside ±0.10 mm
Mean shifted +0.04 / +0.06 / +0.08 mm	Slow tool wear or a wrong offset pushes every part the same way, yet all stay in tolerance
Shared tool offset (correlation 0.6)	Parts from the same tool, setup or batch share most of their error

The slot is sampled the same way but is not truncated. The whole procedure is packaged in a small program, described in Section 8, so anyone can reproduce every number in this paper.

4. Results

4.1 Worst-case versus RSS

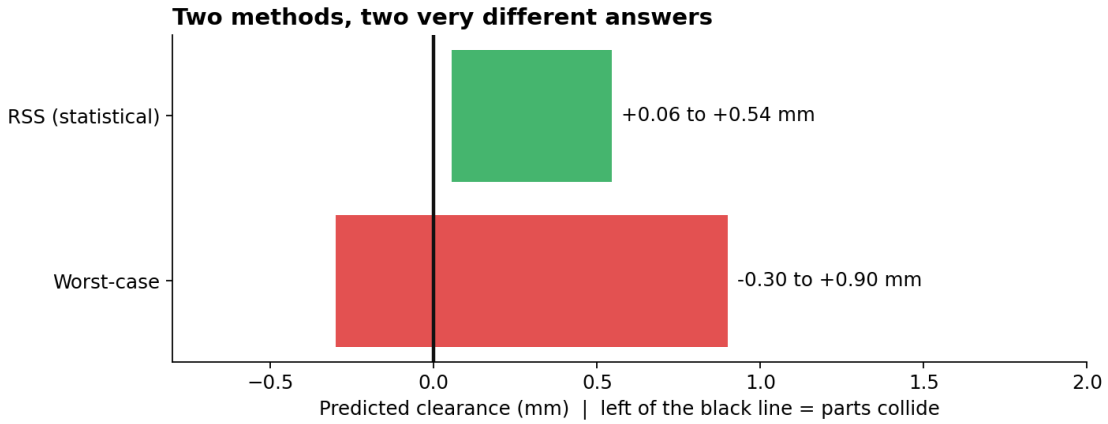


Figure 2. The same stack, two methods, two very different predictions.

Method	Min clearance	Max clearance	Verdict
Nominal	0.30 mm	0.30 mm	looks fine
Worst-case	**−0.30 mm**	+0.90 mm	● can collide
RSS	**+0.055 mm**	+0.545 mm	● always positive

Worst-case says the parts **might not fit**. RSS says they **always fit**. The truth sits between the two and is decided by how the parts are actually made.

4.2 What the simulation says

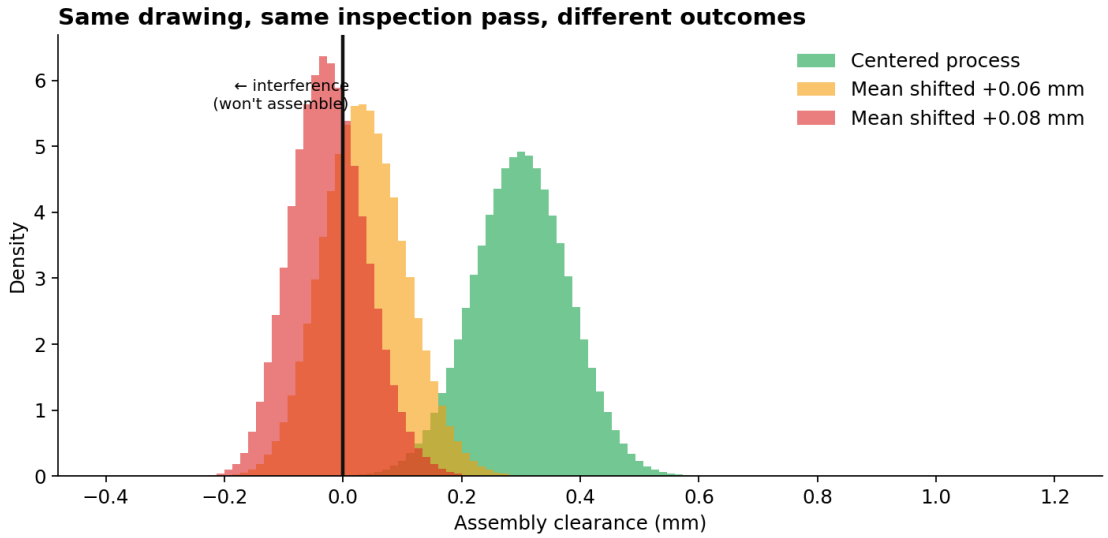


Figure 3. Distribution of assembly clearance. Everything left of the black line is an assembly that will not go together.

Scenario	Mean clearance (mm)	Spread σ (mm)	Assemblies that fail
● Centered process	0.300	0.081	**0.01 %**
● Uniform across the band	0.300	0.133	**1.1 %**
● Mean shifted +0.04 mm	0.114	0.076	**6.6 %**
● Mean shifted +0.06 mm	0.037	0.070	**30.8 %**
● Mean shifted +0.08 mm	−0.023	0.063	**65.5 %**
● Shared tool offset (+0.04, correlated)	0.127	0.122	**15.2 %**

Every row uses parts that all passed their ±0.10 mm inspection.

4.3 The slow slide toward failure

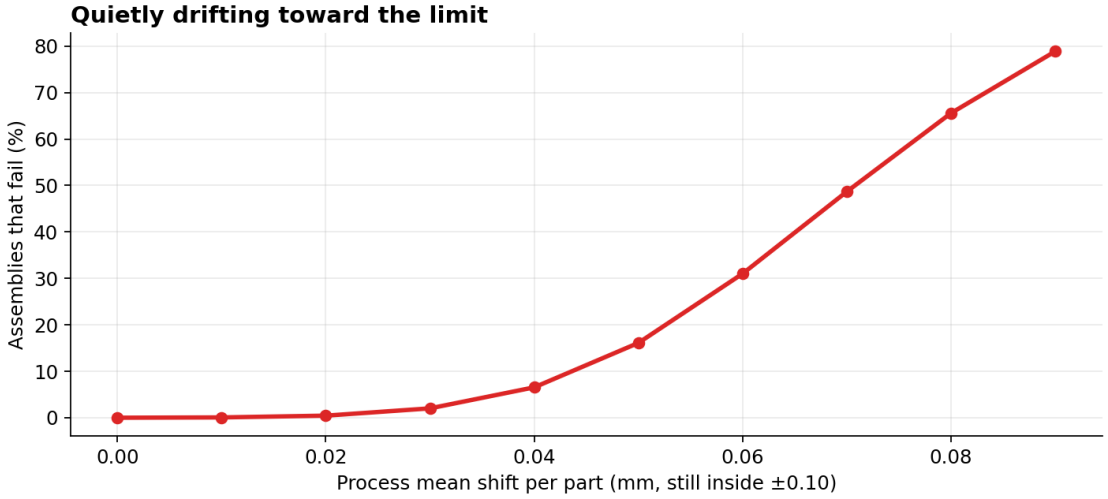


Figure 4. As the process mean drifts toward the upper limit, failures do not rise gently. They accelerate.

Mean shift per part	0.00	0.02	0.04	0.06	0.08	0.09
Assemblies that fail	0.008 %	0.45 %	6.5 %	31 %	66 %	79 %

5. Discussion: Why “Perfect” Parts Fail

1. Tolerances add, but nobody inspects the sum. Each part is judged alone. The stack has no inspector, so the failure appears for the first time at assembly.

2. A part can be “in tolerance” and still be a bad part. A shaft measuring 20.09 mm is *good* on paper. But five parts of 20.09 mm leave no room for the housing. Passing inspection means *within the limits*, not *good for the assembly*.

3. Real processes drift. Tool wear, thermal growth, a work offset typed 0.05 mm off, or a worn insert all push dimensions the same direction. The center of the process moves, and the

RSS “randomness” assumption quietly stops being true. A drift of only **0.06 mm**, still *inside* the tolerance band, makes almost one in three assemblies fail.

4. Errors are rarely independent. Parts cut with the same tool, setup, offset or batch share their errors. The shared-offset scenario widened the spread by about 60% and more than doubled the failure rate compared with independent errors of the same average shift (15.2% against 6.6%).

5. The clearance was designed too small. A 0.30 mm clearance across a 100 mm stack is only **0.3%**. Once the tolerances of six features are added, there is almost nothing left to spend.

6. Worked Example, Step by Step

This section repeats the calculation by hand, so the logic is visible without any software.

Step 1: Write the loop equation. The closing dimension is the slot minus everything inside it:

$$C = S - (P_1 + P_2 + P_3 + P_4 + P_5)$$

Step 2: Nominal value. C = 100.30 − 5 × 20.00 = **0.30 mm**.

Step 3: Worst-case. Add every tolerance: 5 × 0.10 + 0.10 = **0.60 mm**. So C ranges from 0.30 − 0.60 = **−0.30 mm** to 0.30 + 0.60 = +0.90 mm. A negative value means collision.

Step 4: RSS. √(5 × 0.10² + 0.10²) = √0.06 = **0.245 mm**. So C ranges from **+0.055** to +0.545 mm.

Step 5: Add drift. If every part is made *s* mm too large on average, the mean clearance falls by 5 × *s*. A quick estimate is 0.30 − 5*s*. The simulation gives slightly higher values, because inspection throws away the parts that are too large:

Drift s (mm)	Hand estimate 0.30 − 5s	Simulated mean	Assemblies failing
+0.04	+0.100 mm	+0.114 mm	6.6 %
+0.06	+0.000 mm	+0.037 mm	30.8 %
+0.08	−0.100 mm	−0.023 mm	65.5 %

> ***Reading the table:*** *at a drift of +0.06 mm the average assembly has zero clearance. Half the assemblies are already on the wrong side of fitting, before any random variation is counted.*

6.1 Fixing it without tightening tolerances

The cheapest repair is not tighter parts but a bigger slot. Widening the slot nominal from 100.30 to 100.60 mm (clearance 0.60 mm) and re-running the software with the same drifting processes gives:

Scenario	Slot 100.30 mm	Slot 100.60 mm
Mean shifted +0.06 mm	🔴 30.8 % fail	🟢 0 failures in 542,084 assemblies
Mean shifted +0.08 mm	🔴 65.5 % fail	🟢 0 failures in 201,131 assemblies
Shared offset (+0.04, correlated)	🔴 15.2 % fail	🟢 0 failures in 885,774 assemblies

Design clearance is a form of insurance: it costs a little space and buys a lot of robustness.

7. How to Prevent It

Strategy	What it does
Run a stack-up before releasing the drawing	Find the closing dimension and add the loop on paper first
Use both WC and RSS	WC for safety-critical fits; RSS where cost matters and the process is proven stable
Center the process, not just the part	Track the mean of measured parts, not only pass/fail
Add a compensator	A shim, a slotted hole or an adjustable stop absorbs the accumulated error
Reduce the number of links	Fewer parts in the chain means less accumulation
Apply tolerance where it matters	Tighten the one dimension that dominates the stack, and relax the others
Use the right fastener rule	Floating fastener: roughly H = F + T. Fixed fastener: roughly H = F + 2T (F = fastener size, T = positional tolerance)

> ***In CNC terms:*** *when the same work offset or tool-wear value affects every part in a batch, all parts move together. That is exactly the correlated, drifting case that produced the highest failure rates in this study.*

8. The Software: Tolerance Stack-Up Simulator

8.1 What it is

stackup_simulation.py (v1.1) is a small Python program written for this study. It answers one question: *if N identical parts are stacked in a slot, and every part passes inspection, how often does the assembly still fail?* It needs only Python and NumPy, and it runs in a few seconds, even on a phone running Linux.

8.2 How it was used in this paper

1. The dimensions of the example (5 × 20.00 ± 0.10 mm in a 100.30 ± 0.10 mm slot) were entered as inputs.
2. The full study was run once with a fixed random seed (2026), so the results are exactly repeatable.
3. Its printed numbers became Tables 4.2 and 4.3, and its plots became Figures 3 and 4.
4. Extra one-scenario runs (the `--shift` and `--slot` options) produced the repair test in Section 6.1.

8.3 How it works

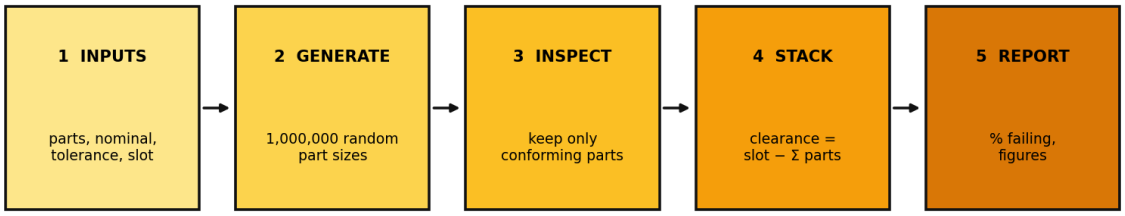


Figure 5. The five stages of the simulator.

Stage	What happens
1. Inputs	Number of parts, nominal size, tolerance, slot size, drift, correlation, sample count
2. Generate	For each of 1,000,000 virtual assemblies, draw a random size for every part (normal, σ = T/3)
3. Inspect	Throw away any assembly containing a part outside its ±T band, exactly like a quality check
4. Stack	Clearance = slot size − sum of the parts. Negative means the parts collide
5. Report	Print mean, spread and the percentage of failures; draw the figures

8.4 The code, explained

The heart of the program is one function that generates the parts. Every part gets its own random error, plus an optional **shared error** that models a common tool offset or batch effect:

Listing 1. Generating the virtual assemblies

```
def sample(mean_shift=0.0, correlated=0.0, dist="normal"):
    sigma = T / 3                       # process capability Cp = 1
    common = rng.normal(0, sigma, M)    # error shared by ALL parts
    parts = []
    for _ in range(N):
        own = rng.normal(0, sigma, M)   # error of this part only
        x = NOM + mean_shift + np.sqrt(1 - correlated) * own \
                             + np.sqrt(correlated) * common
        parts.append(x)
    slot = rng.normal(SLOT_NOM, SLOT_T / 3, M)
    return slot - np.sum(parts, axis=0), np.array(parts)
```

The square-root weights keep each part's total spread equal to σ, whatever the correlation. Only the *relationship between parts* changes. Next, the inspection filter keeps only assemblies that a real quality check would have accepted:

Listing 2. The inspection filter and the failure count

```
def run(**kw):
    c, p = sample(**kw)
    ok = np.all((p >= NOM - T) & (p <= NOM + T), axis=0)   # every part in tolerance?
    c = c[ok]                                               # keep conforming assemblies only
    return c, {"mean": c.mean(), "std": c.std(),
               "fail_pct": np.mean(c < 0) * 100}            # clearance < 0 -> will not fit
```

Finally, the two textbook formulas are computed directly, so the simulation can always be compared with the analytical answer:

Listing 3. The analytical limits

```
wc  = N * T + SLOT_T                  # worst-case: tolerances add
rss = np.sqrt(N * T**2 + SLOT_T**2)   # root-sum-square
print(f"Worst-case range : {nom_clear-wc:+.3f} to {nom_clear+wc:+.3f} mm")
print(f"RSS range        : {nom_clear-rss:+.3f} to {nom_clear+rss:+.3f} mm")
```

8.5 How to use it

Install: Python 3.8 or newer, then `pip install numpy matplotlib` (matplotlib only if you want figures). **Run:**

Terminal

```
python stackup_simulation.py                          # full study + 2 figures
python stackup_simulation.py --no-plots               # numbers only
python stackup_simulation.py --shift 0.06             # one custom scenario
python stackup_simulation.py --shift 0.06 --slot 100.60   # test a redesign
python stackup_simulation.py --parts 8 --nominal 12 --tol 0.05 --slot 96.5
```

Option	Meaning	Default
--parts	Number of stacked parts	5
--nominal / --tol	Part size and ± tolerance (mm)	20.00 / 0.10
--slot / --slot-tol	Slot size and ± tolerance (mm)	100.30 / 0.10
--shift	Run one scenario with this process drift (mm)	off
--corr	Shared-error fraction, 0 to 1 (use with --shift)	0.0
--samples / --seed	Assemblies simulated / random seed	1,000,000 / 2026
--out / --no-plots	Figure folder / skip figures	images / off

8.6 Real output

This is the program's actual output with the default settings, the source of the numbers in Section 4:

Terminal output

```
Nominal clearance : +0.300 mm
Worst-case range  : -0.300 to +0.900 mm
RSS range         : +0.055 to +0.545 mm

Centered process (Cp=1)                     mean=+0.300  std=0.081  FAIL=  0.01 %
Uniform across tolerance band               mean=+0.300  std=0.133  FAIL=  1.07 %
Process mean shifted +0.04 mm               mean=+0.114  std=0.076  FAIL=  6.57 %
Process mean shifted +0.06 mm               mean=+0.037  std=0.070  FAIL= 30.82 %
Process mean shifted +0.08 mm               mean=-0.023  std=0.063  FAIL= 65.47 %
Shared tool offset, shift +0.04             mean=+0.127  std=0.122  FAIL= 15.23 %

Shift sweep (mm -> % fail):
  0.00 ->   0.008 %
  0.01 ->   0.074 %
  0.02 ->   0.453 %
  0.03 ->   2.024 %
  0.04 ->   6.549 %
  0.05 ->  16.114 %
  0.06 ->  31.026 %
  0.07 ->  48.688 %
  0.08 ->  65.581 %
  0.09 ->  78.881 %
```

8.7 Download

The program is free to download and use for learning. If it helps your own work, please cite this paper.

Download stackup_simulation.py

Google Drive · single Python file

Open on Google Drive > stackup_simulation.py

https://drive.google.com/file/d/1Ah_jteal6ScaAFoZLhkVvYVh6UQ79Xn1/view?usp=drivesdk

8.8 Limits of the software

It models a **one-dimensional** stack only. It assumes normally distributed errors and ignores temperature, deflection, surface finish, and angular or positional (GD&T) errors. Treat it as a teaching and checking tool, not a replacement for full 3D tolerance-analysis software.

9. Limitations, Future Work and Conclusion

Limitations. All numbers come from the simulation of one example stack; no physical parts were measured in this version. Real distributions can be skewed or have several peaks, and real assemblies are three-dimensional.

Your turn to measure. A physical version of this study needs only a caliper: measure five identical real objects, then measure the space they must fit into. Compare the *sum of measured sizes* with the predicted stack and see which method, worst-case or RSS, comes closest. The results can become Section 4.4: Field Measurements.

Conclusion

Two perfect parts can refuse to fit because **a tolerance describes one dimension, while a product fails in the sum of many.** Worst-case analysis is too pessimistic, RSS is too optimistic when the process drifts, and the real answer lives in how the parts are actually made.

In this example, the very same drawing gave **0.01% failures with a centered process and 65% failures when the process drifted by 0.08 mm**, without a single part ever going out of tolerance.

> ***The lesson:*** *Don't only ask “is this part within tolerance?” Ask “where inside the tolerance is it, and what does it add up to?”*

References

1. Fischer, B. R. (2011). *Mechanical Tolerance Stackup and Analysis* (2nd ed.). CRC Press.
2. ASME Y14.5, *Dimensioning and Tolerancing*. The American Society of Mechanical Engineers.
3. ISO 286-1, *Geometrical product specifications (GPS): ISO code system for tolerances on linear sizes*.
4. Chase, K. W., and Parkinson, A. R. (1991). A survey of research in the application of tolerance analysis to the design of mechanical assemblies. *Research in Engineering Design, 3*(1), 23–37.
5. Bender, A. (1962). Benderizing tolerances: a simple practical probability method of handling tolerances for limit-stack-ups. *Graphic Science*.
6. Simplexity Product Development. (2022). *The Three Steps of Tolerance Analysis* (whitepaper).

About the Author



MD Ikramul Hoque (Arik)

Maker · CNC Programmer · Developer · Math Coach

1st Merit Position, BKTTC SICIP CNC Trainee
Chattogram, Bangladesh

Arik is a maker from Chattogram who learns by building: on a CNC machine, on a circuit board, in a simulator, in a Linux system running inside a phone, and in busy teaching centers. His work covers CNC programming in Fanuc G-code, CAD modeling, web development and cybersecurity, and he coaches students in mathematics.

He started this research series, *Two Perfect Parts That Refuse to Fit Together* and the experiments after it, to turn everyday engineering puzzles into clear, visual and honest studies that anyone can reproduce. He is preparing to study engineering from 2027.

Contact	
Email	mdikramulhoquearik@gmail.com
Portfolio	IkramulHoque-Arik.github.io
Location	Chattogram, Bangladesh
Status	Open to opportunities · Engineering 2027

ITERATION LAB
DESIGN · SIMULATE · MACHINE

The Iteration Lab
Design · Simulate · Machine Engineering studies, tools and experiments.

How to cite this paper

Hoque, M. I. (Arik). (2026). Two Perfect Parts That Refuse to Fit Together: How individually conforming components fail to assemble when dimensional tolerances accumulate. The Iteration Lab, Chattogram, Bangladesh.